

How torch/ss_torch.py works — the complete account

2026-06-10

Contents

1. The problem being solved	1
2. The equations, in the form the solver uses	2
3. The unknowns: two small neural nets	2
4. The loss, term by term	3
5. Gradient gating: the continuous Gauss–Seidel	6
6. Training pipeline	6
7. Validation (the FD gospel: reference only, never in training)	7
8. Failure modes (all observed, all understood)	8
9. What is and isn’t ported; file map	8

Single-file PyTorch port of the JAX ss/ soft-penalty solver: the stationary Aiyagari equilibrium solved as a PINN, with NO finite-difference solve anywhere in training. This note explains every piece: the economics, the exact equations, both KFE formulations, every loss term and why it exists, the gradient gating, the training schedule, the validation, and the observed failure modes. Companion docs: notes/ss-torch-cheat-audit-2026-06-10.md (audit), ss/config.py + ss/residual.py docstrings (the JAX original this ports).

1. The problem being solved

The EMINN Krusell–Smith model (Gu–Laurière–Merkel–Payne 2024, App. B.1) at its **deterministic steady state**: aggregate TFP is pinned at $z = z_{\text{bar}}$ (no OU shocks), so the economy collapses to the classic Aiyagari/Huggett stationary equilibrium.

Households. Hold wealth $a \in [a_{\text{min}}, a_{\text{max}}] = [1e-6, 20]$, earn labor income $w \cdot l$ with $l \in \{l_1, l_2\} = \{0.3, 1.7\}$ switching by a symmetric Poisson process ($\lambda_1 = \lambda_2 = 0.4$), choose consumption c with CRRA utility $u(c) = (c^{1-\gamma} - 1)/(1 - \gamma)$, $\gamma = 2.1$, discount rate $\rho = 0.05$. Budget: $\dot{a} = s = w \cdot l + r \cdot a - c$.

Borrowing constraint (soft form). Instead of a hard $a \geq 0$, the flow utility carries the paper’s penalty $\psi(a) = -\frac{1}{2}\kappa(a - a_{\text{lb}})^2$ for $a \leq a_{\text{lb}}$ (zero above), with $\kappa = 3$, $a_{\text{lb}} = 1$. Below a_{lb} the penalty’s *gradient* $\psi'(a) = -\kappa(a - a_{\text{lb}}) > 0$ raises the marginal value of wealth, which raises saving, so households endogenously avoid the region — the constraint never literally binds. (The hard-constraint variant of ss/ is NOT ported.)

Firms / prices. Cobb–Douglas $Y = e^z K^\alpha L^{1-\alpha}$ ($\alpha = 1/3$, depreciation $\delta = 0.1$), competitive factor markets:

$$r = \alpha e^z (K/L)^{\alpha-1} - \delta, \quad w = (1-\alpha) e^z (K/L)^\alpha \quad [\text{prices()}]$$

Equilibrium = a triple (policy, distribution, prices) such that: 1. the policy is optimal given prices (HJB); 2. the distribution is stationary under that policy (KFE); 3. prices are the marginal products at the distribution's own aggregates $K = \sum_j \int a g_j da$, $L = \sum_j \int l_j g_j da$ (market clearing).

2. The equations, in the form the solver uses

2.1 HJB in marginal-value form

The solver never represents V ; it works with $W = \partial_a V$ (as the whole repo does). Start from the stationary HJB

$$\rho V = \max_c u(c) + \psi(a) + (w \cdot l + r \cdot a - c) \partial_a V + \lambda_j (V_{\text{other}} - V)$$

The FOC gives $u'(c) = W$, i.e. $c = W^{-1/\gamma}$ (this is why $W > 0$ is structural). Differentiate the HJB w.r.t. a ; the $\partial c / \partial a$ terms cancel by the envelope theorem ($u'(c)c' - c'W = 0$), leaving

$$0 = (\rho - r + \lambda_j) W - s \partial_a W - \lambda_j W_{\text{other}} - \psi'(a) \quad [\text{the HJB residual}]$$

with $s = w \cdot l_j + r \cdot a - W^{-1/\gamma}$. All z -derivative terms and the distribution-coupling term L_g of the full master equation vanish at the deterministic steady state — that is exactly what makes the SS problem self-contained. One extra structural fact used later:

$$\partial_a s = r - c'(a) = r + (1/\gamma)(c/W) \partial_a W$$

2.2 Stationary KFE

For each labor state j (writing $\tilde{j}$ for the other state):

$$0 = -\partial_a(s_j g_j) + \lambda_{\{\tilde{j}\}} g_{\{\tilde{j}\}} - \lambda_j g_j, \quad \text{zero flux at } a_{\min}, a_{\max}$$

Two exact integral consequences (both used as losses): - **Labor-mass balance** (integrate over a ; boundary fluxes vanish): $\lambda_1 M_1 = \lambda_2 M_2$ where $M_j = \int g_j$. With $\lambda_1 = \lambda_2$ this forces the 50/50 split, hence $L = 1$. - **Aggregate-saving identity** (multiply by a , integrate, sum over j ; the jump terms cancel in the sum because switching l only relabels mass): $dK/dt = \int \sum_j s_j g_j da = 0$ — at stationarity the population saves nothing in aggregate.

3. The unknowns: two small neural nets

Both are plain MLPs in $(a_{\text{scaled}}, \text{onehot}(l))$ — 3 inputs, 4 tanh hidden layers $\times$ width 64, linear head, Glorot/Xavier init, a affinely scaled to $[-1, 1]$. Float32, seeds fixed at import. `both(net, a)` evaluates the two labor states in one batched forward (`a.repeat(2)` against stacked one-hot, reshaped to $(B, 2)$).

3.1 WNet — the policy side

$$W(a, l_j) = \text{softplus}(\text{MLP}(a, l_j))$$

Softplus enforces $W > 0$ (marginal utility is positive; $c = W^{-1/\gamma}$ requires it). This also blocks the *parametrization* form of the $W \rightarrow 0$ cheat — but note $W \rightarrow 0$ is an *attractor* the optimizer can still approach (W small $\Rightarrow c$ huge $\Rightarrow s$ very negative $\Rightarrow$ transport term washes the HJB residual out); what actually guards against it is the pretrain LEVEL (§6) and the shape penalty. Audit check D1: trained $\min W = 0.19$, nowhere near the corner.

3.2 GNet — the distribution side

$$\begin{aligned} f(a, l_j) &= \text{MLP}(a, l_j) - \text{softplus}(\beta_j) \cdot (a - a_{\min}) && [\text{raw log-density}] \\ g(a, l_j) &= \exp(f(a, l_j) - \log Z) && [\text{normalized density}] \end{aligned}$$

Three deliberate choices:

- **Log-space.** Positivity for free, and densities spanning 4+ orders of magnitude (peak ~ 0.5 vs tail $\sim 1e-5$) become $O(1)$ quantities the net can represent.
- **Trainable tail slope β_j .** The true stationary density decays $\sim$ exponentially. A bounded tanh-MLP in log space cannot steepen its own tail fast enough — the KFE residual in the tail is proportional to the tiny local density, so the gradient signal vanishes exactly where the error lives; meanwhile the tail has a long lever arm into K (mass at $a \approx 15$ is $3\times$ the mean). The explicit $-\text{softplus}(\beta_j)(a - a_{\min})$ term makes the decay *rate* a single parameter per labor state that receives gradient from every tail node. Initialized at the pretrain prior's rate $1/\bar{a}$ via softplus-inverse.
- **Normalization by construction.** $\log Z = \text{logsumexp}$ over (nodes $\times$ states) of $(f + \log w_q)$ on a FIXED 400-pt uniform trapezoid quadrature (quad_rule), so $\sum_j \int g = 1$ to quadrature error *identically in the weights*. This removes the KFE's scale invariance (any multiple of a stationary g is stationary) without a normalization penalty the optimizer could trade off against other terms. Audit D2: mass = 1.0000.

3.3 Prices by construction

Every loss evaluation recomputes, from the g-net's current weights:

$$f_q \rightarrow \log Z \rightarrow g_q \text{ (400}\times\text{2)} \rightarrow K = \sum a \cdot w_q \cdot g, \quad L = \sum l \cdot w_q \cdot g \rightarrow r, w = \text{prices}(K, L)$$

There is no stored price variable and no outer bisection: capital/labor market clearing holds *identically at every step* because r, w are defined as marginal products at the g-net's own aggregates. The classical nested algorithm (solve a full partial-equilibrium problem per candidate r , bisect; see `fd_solve`) is replaced by one flat loop in which the distribution shifting IS the price adjustment. This relies on the Aiyagari fixed point being *stable* (K too high $\rightarrow r$ low $\rightarrow$ saving down $\rightarrow$ mass flows down $\rightarrow K$ falls); an unstable GE feedback would break the joint scheme.

4. The loss, term by term

`make_loss(p)` builds a closure over the quadrature grid; each call does:

4.1 HJB residual (trains the W-net)

At `n_col = 256` fresh random collocation points per step (§6.2), both labor states:

$$L_{\text{hjb}} = \text{mean}[((\rho - r + \lambda_j)W - s \cdot \partial_a W - \lambda_j \cdot W_{\text{other}} - \psi'(a))^2]$$

- $\partial_a W$ by **exact autograd**: the batch is `a.repeat(2)` with `requires_grad`, one forward, one `torch.autograd.grad(W.sum(), a, create_graph=True)`. The `.sum()` trick is valid because the Jacobian is diagonal (each output depends on its own input only); `create_graph=True` makes the derivative itself differentiable so the loss can train through it.
- The policy inside is **live** (s depends on W through $c = W^{-1/\gamma}$) — the W -net is being asked to satisfy its own FOC-consistent equation.
- Prices are **detached** (§5).
- This term is fully mesh-free in both KFE modes.

4.2 Shape penalty (trains the W -net)

$$L_{\text{shape}} = \text{mean}[\max(\partial_a W, 0)^2]$$

The paper’s monotonicity prior: marginal value decreases in wealth (V concave). Costs nothing when satisfied (it is, after warm-up: the term reads 0.0 throughout) but blocks sign-flipped/oscillating W configurations during the transient, and is one of the two guards against drifting toward the $W \rightarrow 0$ corner. Audit D1: $\partial_a W < 0$ everywhere on a 2000-pt grid.

4.3 KFE — form “fv” (DEFAULT; trains the g -net)

The paper’s conservative upwind finite-volume operator (eq B.1, `kfe_drift`) applied to the g -net’s masses on the quadrature grid:

$$\begin{aligned} m &= g_q \cdot da && \text{(cell masses from the net's densities)} \\ F_{\{m+1/2, j\}} &= s^+_{\{m, j\}} m_{\{m, j\}} + s^-_{\{m+1, j\}} m_{\{m+1, j\}} && \text{(upwind face flux)} \\ F \text{ at the outer faces} &= 0 && \text{(reflecting / zero-flux boundaries)} \\ \mu_{\{m, j\}} &= -(F_{\{m+1/2\}} - F_{\{m-1/2\}})/da + \lambda_{\{j\}} m_{\{m, j\}} - \lambda_j m_{\{m, j\}} \\ L_{\text{kfe}} &= \text{mean}[(\mu/da)^2] \end{aligned}$$

Crucial reading: this is **not an FD solve**. No generator matrix is assembled, no linear system is solved; `kfe_drift` is an algebraic map (shifts/clamps/multiplies) from the net’s values to “the rate at which mass would flow if this density evolved under this policy.” Stationarity $\Leftrightarrow \mu = 0$, and the gradient flows through m into the g -net. The FD *solver* exists in the file only as the validation reference (audit Test A: scrambling it leaves training bit-identical). Audit B2: this operator equals the FD generator A^T to $3e-10$ — fv training enforces the same discretized equation FD solves, but finds g by gradient descent.

Why conservative/upwind matters: the column sums of the flux differences telescope to the boundary faces (which are zero), so total mass is conserved **for any policy**, the zero-flux BCs are built in, and the two structural cheats of the strong form (§4.4) are impossible by construction. The μ/da scaling makes the residual a density-rate, so the loss magnitude is grid-resolution-independent.

The policy s_q on the grid is computed from the W -net under `no_grad` (§5).

4.4 KFE — form “strong” (strong CLI arg; trains the g -net)

PURE mesh-free: the KFE enforced by exact autodiff at the random collocation points,

$$L_kfe = \text{mean}[(-(s \cdot \partial_a g + g \cdot \partial_a s) + \lambda_{\{j\}} g_{\{j\}} - \lambda_j g_j)^2]$$

with $\partial_a g = g \cdot \partial_a f$ (autograd through the g-net, same repeat-trick as 4.1) and the policy pair $(s, \partial_a s = r + (1/\gamma)(c/W)\partial_a W)$ computed from the HJB block’s live values and then **detached**. No discretized differential operator anywhere — “mesh-free” precisely means the PDE is enforced by exact autodiff at sampled points; integral identities may still be evaluated by quadrature of net *values* (normalization already requires that), which involves no stencils.

The pointwise residual alone is NOT well-posed in practice. Three exact identities close the gaps; each kills a failure mode diagnosed in the JAX runs (`ss/residual.py`):

1. **Pointwise total-flux identity** $L_flux = \text{mean}[(\sum_j s_j g_j)^2]$. Summing the two KFE equations cancels the jump terms, so the strong residual only forces $\partial_a(\text{total flux}) = 0$ — total flux = *any constant* has zero residual. The nonzero-constant family looks like $g \sim 1/|s|$ and is the wrong distribution (JAX run 1 found exactly this: K drifted to ~ 11 , $r < 0$, near-zero KFE loss). Forcing the flux to be 0 *pointwise* kills the family: $F_1 = -F_2 = \text{const} \neq 0$ would force $\lambda_1 g_1 = \lambda_2 g_2$ AND $s_1 g_1 = -s_2 g_2$ pointwise $\Rightarrow g_1(s_1 + s_2 \lambda_1/\lambda_2) = 0 \Rightarrow$ generically $g = 0$, excluded by normalization. (In fv form the boundary faces are hard-zero, so the constant cannot be nonzero — immune.)
2. **Labor-mass balance** $L_mass = (\lambda_1 M_1 - \lambda_2 M_2)^2$ with $M_j = \sum w_q \cdot g_q[:, j]$. An MSE-imperfect pointwise residual does not pin the labor split (JAX run 3 settled at $L \approx 0.3$, $r \approx -0.05$ — a pseudo-equilibrium). In fv form the balance telescopes exactly inside the operator — built in.
3. **Aggregate-saving identity** — same term as §4.5, ramped; pins K in both forms.

Plus **zero-flux endpoint penalties** $L_bc = \sum_j (s \cdot g)^2|_{\{a_min\}} + (s \cdot g)^2|_{\{a_max\}}$ (the BCs the fv operator gets for free), and an extra **top collocation band** (§6.2): near a_max the employed’s $s \approx 0$, so the transport term $s \cdot \partial_a g$ barely transmits information — the region needs explicit sampling density.

4.5 Aggregate-saving identity (both forms; trains the g-net)

$$L_agg = (\sum_{\{nodes, j\}} w_q \cdot s_j \cdot g_j)^2 \quad (\text{one scalar, squared})$$

Why it exists at all: the pointwise KFE MSE plateaus at a floor set by per-batch policy jitter ($\sim 1.4e-7$ in the JAX fv runs), and a 10×-overweight TAIL — worth +0.2 on K through the $a \approx 15$ lever arm — hides *below* that floor. The identity is the first moment of the stationary KFE ($dK/dt = 0$), exactly the K-relevant functional the pointwise residual barely sees. Its gradient w.r.t. g_i is $\propto s_i$: it moves mass *from dissaving regions (the fat tail) toward savers* — surgical, unlike a tail-reweighting of the MSE. (Two alternatives failed in JAX: a relative pointwise residual diverges on near-zero-density points; a trainable tail slope alone gets no gradient against the MSE floor.)

Why it is RAMPED (weight 0 until 20% of n_iter , linear to full at 50%): switched on from step 0 it acts on the garbage pretrain-transient policy and crushes all mass to the bottom — K locks at ~ 1.2 – 1.4 . This is not folklore: both of this port’s smoke runs ($n_iter = 1000$, so the ramp engages at step 200 while $HJB \approx 0.16$) reproduced the crush exactly. By 8k/40k steps the policy is sane ($HJB \sim 1e-3$) and the identity is purely stabilizing.

4.6 Total

$$\text{total} = L_{\text{hjb}} + L_{\text{kfe}} + L_{\text{flux}} + \text{agg_w} \cdot L_{\text{agg}} + L_{\text{mass}} + L_{\text{bc}} + L_{\text{shape}}$$

All weights are 1 (the ramp factor agg_w is the only schedule); in fv mode $L_{\text{flux}} = L_{\text{mass}} = L_{\text{bc}} = 0$ identically.

5. Gradient gating: the continuous Gauss–Seidel

One joint loss, one backward pass, one Adam step over BOTH nets simultaneously — there is no alternation. The Gauss–Seidel character comes from exactly two detachments:

- r, w are **detached** right after computation $\Rightarrow \partial L_{\text{hjb}} / \partial (\text{g-net}) = 0$. The g-net cannot bend aggregates/prices to flatter the HJB residual; the W-net solves “the HJB at the current prices.”
- the policy values (s_q on the grid; $s, \partial_a s$ at collocation in strong mode) are **detached** in every g-side term $\Rightarrow \partial (L_{\text{kfe}} + L_{\text{flux}} + L_{\text{agg}} + L_{\text{bc}}) / \partial (\text{W-net}) = 0$. The W-net cannot bend the policy to flatter the KFE; the g-net solves “the KFE under the current policy.”

Each net descends only its own equation given the other’s current values — a continuous Gauss–Seidel iteration on the Aiyagari fixed point, which converges because that fixed point is economically stable (§3.3). Audit Test E verified both cross-gradients are exactly 0.0 at the trained point. (The HJB’s own policy stays LIVE — that is the net’s optimality condition, not a coupling channel.)

6. Training pipeline

6.1 Pre-training (1000 steps, Adam @ $3e-4$) — model-implied targets, never FD

- $f \rightarrow -a/\bar{a}$ ($\bar{a} = 5$): a generic decreasing exponential — the right *qualitative* shape with prior capital $K_0 = \int a \cdot e^{-a/\bar{a}} / Z \approx 4.63$.
- $W \rightarrow u'(w_0 \cdot l + r_0 \cdot a)$: the marginal value of the **zero-saving policy** at the prices (r_0, w_0) implied by the pretrain density’s own ($K_0, L_0 = 1$).

Why these two targets specifically: - **Level at the constraint**. The HJB near $a_{\min}$ must cancel $\psi'(a_{\min}) \approx \kappa \cdot a_{\text{lb}} = 3$ against $(\rho - r + \lambda)W \approx 0.3 \cdot W$, which needs $W(a_{\min}, l_1) \approx u'(w_0 l_1) \approx 10$. The finite-agent warm-start shape ($a_0 + a$) ^{$-\eta$} is $\sim 30\times$ too low there and traps the W-net in an HJB-vs-shape-penalty standoff at the constraint. - **Zero-saving start**. $c = \text{cash} \Rightarrow s \approx 0$ everywhere, so during the early joint phase the g-net is not chasing the stationary distribution of a garbage policy. - **Equal quality for both nets** (the repo’s warm-start-all-nets-equally rule): warm-starting one net much better than the other lets the better one overfit the worse one’s noise.

Pretraining is deliberately crude: post-pretrain the gate metric is ~ 44.5 . Residual training does all the work ($44.5 \rightarrow 3.9e-3$) — audit point C.

6.2 Collocation sampling (fresh every step)

$n_{\text{col}} = 256$ points, each scored at both labor states: - 50% uniform on $[a_{\min}, a_{\max}]$ (global coverage); - 50% uniform on $[a_{\min}, a_{\text{focus}} = 6]$ — the focus band over the constraint + density peak, where the KFE has all its curvature; - strong mode only: 10% carved out of the uniform share for $[a_{\max} - 2, a_{\max}]$ (the weak-transport zone, §4.4).

The quadrature grid (normalization/aggregates/identities/FV) is FIXED — sampling never moves it.

6.3 Optimization

Single Adam over the union of both nets' parameters; LR linearly decayed $3e-4 \rightarrow 1e-6$ over $n_iter = 40,000$ steps (set per-step on the param group). The agg-identity ramp runs over steps [8k, 20k]. Every $n_iter/20$ steps: log the loss components and the validation gate (cheap: one dense no-grad forward; consumes no RNG, so monitoring cannot perturb training — audit A). ~9 min CPU (fv), ~13 min (strong; the extra autodiff through the g-net).

7. Validation (the FD gospel: reference only, never in training)

7.1 The FD reference (`_fd_inner`, `fd_solve`)

Standard Achdou et al. (2022a) upwind scheme on a uniform 400-pt grid: forward/backward one-sided differences of V , upwind selection by the sign of the implied saving, state-constraint BCs ($\partial V_B(a_{\min}) = u'(\text{cash})$, $\partial V_F(a_{\max}) = u'(\text{cash})$), implicit-Euler pseudo-timestepping ($dt = 1000$) of the 800×800 linear system to convergence; then the stationary distribution as the null vector of the generator transpose (replace one row with the normalization). Outer bisection on r : invert the firm FOC for K_r , solve the PE problem, compare $K_g = \int a \cdot g$ with K_r , halve the bracket (~30–60 PE solves).

7.2 The gate

$\|s_{NN} - s_{FD}\|_{\infty}$ on a DENSE 1000-pt grid at **matched FD-equilibrium prices**: the dense FD policy is recomputed at fixed (r_{fd} , w_{fd}) (`fd_gate`; solving the HJB on the dense grid resolves the kink that interpolating a coarse policy would smear), and the NN saving is $s = w_{fd} \cdot l + r_{fd} \cdot a - c_{NN}$. Since $c_{NN} = W^{\{-1/\gamma\}}$ does not depend on prices, passing requires BOTH the right policy shape AND the right equilibrium — if $r_{nn} \neq r_{fd}$ the W-net solved a different problem and the comparison blows up. Gate: $\varepsilon = 1e-2$.

Resolution matching: the FD reference deliberately uses $n_a = 400 = n_{quad}$, because the FD equilibrium K is grid-sensitive (5.27 @ 93 pts, 5.10 @ 400, 5.07 @ 800) and the SS solver bakes its OWN equilibrium into W — gating against a coarser FD would measure discretization mismatch, not solver error. Notably the trained $K_{nn} = 5.0706$ lands on $FD(800) = 5.0730$, closer to the continuum than its own validation reference: the mesh-free HJB side pulls the equilibrium toward the continuum solution.

7.3 Results (this machine, CPU, seed 0 unless noted)

run	gate	K_{nn}	r_{nn}	reference
fv (default)	3.9e-3 PASS	5.0706	0.01295	$K_{fd} = 5.0994$, $r_{fd} = 0.01251$
strong	8.2e-3 PASS	5.0384	0.01342	(noisier late phase, see §8)

run	gate	K_nn	r_nn	reference
fv, prior $\bar{a} = 2.5$	4.9e-3 PASS	5.1115	0.01253	audit C: prior-independence
fv, seed 1	6.6e-3 PASS	5.0523	0.01314	matches JAX seed spread (4.4e-3 / 7.9e-3)

8. Failure modes (all observed, all understood)

- **K-crush by the agg identity** (both forms): engage the identity on a garbage policy and it drains all mass to the bottom, $K \rightarrow \sim 1.2\text{--}1.4$, and the run never recovers (the HJB then equilibrates to the wrong frozen prices). Cure: the ramp (§4.5). Any smoke run short enough that the ramp hits an unconverged policy WILL fail the gate — by design, not a bug.
- **Const-flux cheat family** (strong only): $g \sim 1/|s|$, zero pointwise residual, wrong answer. Cure: the pointwise total-flux identity. Impossible in fv.
- **Labor-split drift** (strong only): pointwise MSE tolerates a wrong M_1/M_2 ; the equilibrium drifts through L. Cure: the mass-balance identity. Impossible in fv.
- **Tail overweight / K bias** (both): hides below the KFE MSE floor. Cure: the agg identity — the trainable tail slope.
- **W→0 attractor**: approached (never reached, softplus) if the W-net’s level near the constraint is initialized far too low. Cure: the pretrain level (§6.1) + shape penalty.
- **Strong-form late-phase noise**: the gate metric oscillates $4e-3 \leftrightarrow 1.2e-2$ over the last 15k steps (briefly failing at 34k/38k, final $8.2e-3$) — the price of losing the FV operator’s built-in conservation. A different seed could land above the gate; same caveat class as the JAX variant.

9. What is and isn’t ported; file map

Ported: the ss/ soft-penalty BASELINE (fv) + VARIANT(strong), behavior-faithful (same calibration, recipe, ramp, targets, identities). NOT ported: --hard-constraint (atoms at $a_{\min}$, boundary singularity exponent, weak/CDF-form KFE, EMA/Polyak stabilizers).

section	functions
§1 calibration + numerics	Par
§2 model math	inv_u_prime, u_prime, prices, soft_penalty_grad, kfe_drift
§3 FD reference (validation ONLY)	_fd_inner, fd_solve, fd_gate
§4 nets + quadrature	MLP, WNet, GNet, both, quad_rule, nn_equilibrium
§5 loss	make_loss (fv + strong branches), draw_collocation
§6 gate + training	saving_linf, train (pretrain + joint phases)
§7 plots	plot_all → plots/ss_torch{,_strong}.png
main	CLI: bare = fv 40k; strong = mesh-free; integer = n_iter override

Checkpoints: torch/ss_torch{,_strong}_state.pt (torch.save of both state dicts).